CHAPTER 25

Language Reference: Objects and Classes

In this second chapter of language reference, we'll cover the syntax of Ruby Objects and classes.

Method Definition

def *defname* ‹(‹, *arg*›*) ›
  body
end

def *defname* ‹(‹, *arg*›*)› = body

defname ← *methodname* | *expr.methodname*

defname contains the name of the method and optionally an object defining context in which it is valid.

A *methodname* is either a redefinable operator (see Table 13, Ruby operators (high to low precedence), on page 455) or a name. If *methodname* is a name, it should start with a lowercase letter (or underscore) optionally followed by uppercase and lowercase letters, underscores, and digits. A *methodname* may optionally end with a question mark (?), exclamation point (!), or equal sign (=). The question mark and exclamation point are simply part of the name. The equal sign is also part of the name but additionally signals that this method may be used as an lvalue (see the description of writeable attributes on page 38).

A method can be written over multiple lines, in which case the method ends with a matching `end` keyword. A single line method can be written with the method name followed by a space, followed by an = followed by the one-line body. Note that the space before the equals sign is important to distinguish this from a method name ending in =. Because this method form does not need a matching `end` it is sometimes referred to as an "endless method."

The statement which creates the method returns the method name as a symbol. This is useful for interacting with Ruby methods that take a symbol expecting it to be a method name, as in `private def foo = 1 + 1`.

A method definition using an unadorned method name within a class or module definition creates an instance method. An instance method defined in a class may be invoked only by sending its name to a receiver that is an instance of the class that defined it, or one of that class's subclasses. An instance method defined in a module may be invoked by a receiver